

RoadSense PH — TRANSFERSS + LOCKED BASELINE

Generated: 2025-12-28 16:47:07.026383

TRANSFERS.md

LOCKED — LGU Map OPS Baseline (Stable)

Date: 2025-12-26 15:56

Scope: `app/lgu/map.base.tsx`

GOALS Alignment

- LGU-first operations board (not consumer navigation).
- Severity-based prioritization (LOW/MED/HIGH).
- Resolved hazards preserved as historical reference.

Locked UX Rules

1) **Placing Mode Single Source of Truth**

- Long-press on map is primary entry to placing mode.
- Report button is a secondary shortcut.
- Placement confirms hazard at the **center pin** using `coordinateForPoint`.

2) **Resolved Must Be Obvious at a Glance**

- Resolved markers are **gray** + **low opacity** + visually de-emphasized.
- Active markers use severity colors.

3) **CRITICAL BUG FIX — Marker State Bleed Prevention**

- `react-native-maps` can reuse marker views.
- We force remount by using marker key: `key = `${id}-${status}``
- This prevents "wrong hazard changes color" after resolve/delete.

4) **Recenter is Non-Negotiable (LGU Field Ops)**

- Dedicated floating recenter button restored and kept visible.
- One-tap returns to user location.

Stability Rule (Permanent)

- Once a feature is proven stable and correct, treat it as **LOCKED**.
- Do not refactor/replace that area without explicit approval and a rollback plan.

LOCKED — LGU Map Baseline v2 (DO NOT TOUCH)

Date: `'"$ts"'`

Files:

- `app/lgu/map.base.tsx`
- `app/lgu/map.base.v2.locked.tsx`

Locked Features:

- Placing mode uses center pin + `coordinateForPoint` (stable UX).
- Recenter restored + high-visibility button (LGU field ops).
- Resolved hazards are visually de-emphasized (gray + low opacity).
- Marker state-bleed fix: key uses `id-status` (`${id}-${status}`) to prevent wrong color changes after resolve/delete.

Rule:

- If any change is needed, fork from `v2.locked` and keep rollback path.

GOALSS.md

GOALSS — RoadSense PH LGU Master Scope (LOCKED v1)

Purpose

RoadSense PH is an LGU-first road hazard intelligence system designed for field operations, prioritization, accountability, and reporting.

System Components

```

### Mobile App (Field Operations)
- Structured hazard capture using center-pin placement
- Severity levels: HIGH / MED / LOW
- Hazard type selection
- Local-first storage (AsyncStorage)
- Resolve and delete flows with accountability
- Designed for speed and reliability in the field

### LGU Dashboard (Office Operations)
- Read-only dashboard for LGU staff
- Displays all reported hazards with:
  - Location
  - Severity
  - Type
  - Status (Active / Resolved)
- Severity-based prioritization
- Historical visibility of resolved hazards
- Printable reports for meetings, audits, and documentation

### Data Flow
- Mobile app pushes hazard data to Supabase (fire-and-forget)
- Dashboard reads from Supabase only
- No realtime synchronization
- No public write access

## Explicitly Out of Scope (v1)
- Barangay or district aggregation
- Consumer navigation features
- Public reporting portals
- Realtime updates or subscriptions
- Predictive analytics or AI
- Any redesign of the LGU map UX

## Product Philosophy
- LGU-first, operations-focused
- Preserve solved work
- Avoid feature creep
- Optimize for adoption, not novelty
- Ship stable, pitch fast

```

This document defines the locked scope of GOALSS v1.
Any changes require explicit scope revision.

app/lgu/map.base.tsx

```

import React, { useEffect, useMemo, useRef, useState } from "react";
import { View, Text, Pressable, Dimensions, Platform } from "react-native";
import MapView, { Marker, Region, MapPressEvent } from "react-native-maps";
import * as Location from "expo-location";
import {
  fetchHazards,
  createHazard,
  resolveHazard,
  deleteHazard,
  Hazard,
} from "@lib/hazards";
/* ----- LGU OPS CONFIG ----- */
const DEFAULT_REGION: Region = {
  latitude: 14.5995,
  longitude: 120.9842,
  latitudeDelta: 0.08,
  longitudeDelta: 0.08,
};
const SEVERITIES = [
  { key: 3, label: "HIGH", bg: "#ef4444" },
  { key: 2, label: "MED", bg: "#f59e0b" },
  { key: 1, label: "LOW", bg: "#10b981" },
] as const;

```

```

const HAZARD_TYPES = [
  { key: "pothole", label: "Pothole" },
  { key: "flood", label: "Flood" },
  { key: "crack", label: "Crack" },
  { key: "debris", label: "Debris" },
  { key: "construction", label: "Construction" },
] as const;
type HazardType = (typeof HAZARD_TYPES)[number]["key"];
function severityColor(sev: number) {
  if (sev === 3) return "#ef4444";
  if (sev === 2) return "#f59e0b";
  return "#10b981";
}
function safeStatus(h: any): "reported" | "resolved" {
  return h?.status === "resolved" ? "resolved" : "reported";
}

function isValidCoord(h: any) {
  const lat = Number(h?.latitude);
  const lng = Number(h?.longitude);
  return Number.isFinite(lat) && Number.isFinite(lng);
}
function shortTypeLabel(t: string) {
  const found = HAZARD_TYPES.find((x) => x.key === t);
  return found ? found.label : String(t || "Hazard");
}
/* ----- COMPONENT ----- */
export default function LGUMap() {
  const mapRef = useRef<MapView | null>(null);
  const [region, setRegion] = useState<Region>(DEFAULT_REGION);
  const [userCoord, setUserCoord] =
    useState<{ latitude: number; longitude: number } | null>(null);
  const [hazards, setHazards] = useState<Hazard[]>([]);
  const [selectedId, setSelectedId] = useState<string | null>(null);
  const [isMutating, setIsMutating] = useState(false);
  // Center-pin placement mode (LOCKED)
  const [placementMode, setPlacementMode] = useState(false);
  const [severity, setSeverity] = useState<1 | 2 | 3>(2);
  const [type, setType] = useState<HazardType>("pothole");
  // LGU filters
  const [severityFilter, setSeverityFilter] = useState<Record<number, boolean>>({
    3: true,
    2: true,
    1: true,
  });
  const [typeFilter, setTypeFilter] = useState<Record<HazardType, boolean>>({
    pothole: true,
    flood: true,
    crack: true,
    debris: true,
    construction: true,
  });
  const [showResolved, setShowResolved] = useState(true);
  async function safeReloadHazards() {
    try {
      const next = await fetchHazards();
      {
        const arr = Array.isArray(next) ? next : [];
        const normalized = arr.map((h:any) => ({
          ...h,
          severity: (h?.severity === "HIGH" ? 3 : h?.severity === "MEDIUM" ? 2 : h?.severity === "LOW" ? 1 :
            h?.severity),
        }));
        setHazards(normalized as any);
      }
    } catch (e) {
      console.warn("[LGUMap] fetchHazards failed", e);
      setHazards([]);
    }
  }
}

```

```

}
// Location bootstrap + initial load
useEffect(() => {
  (async () => {
    try {
      const { status } = await Location.requestForegroundPermissionsAsync();
      if (status === "granted") {
        const loc = await Location.getLastKnownPositionAsync({});
        if (loc?.coords) {
          const r: Region = {
            latitude: loc.coords.latitude,
            longitude: loc.coords.longitude,
            latitudeDelta: 0.06,
            longitudeDelta: 0.06,
          };
          setUserCoord({ latitude: r.latitude, longitude: r.longitude });
          setRegion(r);
        }
      } catch {
        // keep default region
      }
      await safeReloadHazards();
    })();
  }, []);
  const selected = useMemo(() => {
    if (!selectedId) return null;
    return hazards.find((h: any) => String(h.id) === String(selectedId)) ?? null;
  }, [selectedId, hazards]);
  // Clear selection if it disappears
  useEffect(() => {
    if (selectedId && !hazards.some((h: any) => String(h.id) === String(selectedId))) {
      setSelectedId(null);
    }
  }, [hazards, selectedId]);
  // LOCKED: single source of truth
  function enterPlacingMode() {
    setSelectedId(null);
    setPlacementMode(true);
  }
  function cancelPlacement() {
    setPlacementMode(false);
  }
  async function confirmPlacement() {
    if (!mapRef.current || isMutating) return;
    try {
      setIsMutating(true);
      const { width, height } = Dimensions.get("window");
      // Best-effort: center of the screen. Good enough for ops.
      const coord = region;
      await createHazard({
        latitude: coord.latitude,
        longitude: coord.longitude,
        severity,
        type,
      });
      await safeReloadHazards();
    } catch (e) {
      console.warn("[LGUMap] confirmPlacement failed", e);
    } finally {
      setIsMutating(false);
      setPlacementMode(false);
    }
  }
  function onMarkerPress(id: string) {
    if (isMutating || placementMode) return;
    setSelectedId(id);
  }
  async function resolveSelected() {

```

```

if (!selected || isMutating) return;
try {
  setIsMutating(true);
  await resolveHazard(String((selected as any).id));
  await safeReloadHazards();
} catch (e) {
  console.warn("[LGUMap] resolveSelected failed", e);
} finally {
  setIsMutating(false);
}
}
async function deleteSelected() {
  if (!selected || isMutating) return;
  const id = String((selected as any).id);
  setSelectedId(null);
  try {
    setIsMutating(true);
    await deleteHazard(id);
    await safeReloadHazards();
  } catch (e) {
    console.warn("[LGUMap] deleteSelected failed", e);
    await safeReloadHazards();
  } finally {
    setIsMutating(false);
  }
}
function recenter() {
  if (!mapRef.current || !userCoord) return;
  const r: Region = {
    latitude: userCoord.latitude,
    longitude: userCoord.longitude,
    latitudeDelta: 0.06,
    longitudeDelta: 0.06,
  };
  setRegion(r);
  mapRef.current.animateToRegion(r, 350);
}
const hazardsStable = useMemo(() => {
  const arr = [...hazards] as any[];
  arr.sort((a, b) => String(a.id).localeCompare(String(b.id)));
  return arr;
}, [hazards]);
const visibleHazards = useMemo(() => {
  return hazardsStable.filter((h: any) => {
    const st = safeStatus(h);
    if (!showResolved && st === "resolved") return false;
    if (!severityFilter[h.severity]) return false;
    if (!typeFilter[h.type as HazardType]) return false;
    return true;
  });
}, [hazardsStable, severityFilter, typeFilter, showResolved]);
const counts = useMemo(() => {
  let high = 0, med = 0, low = 0;
  let active = 0, resolved = 0;
  for (const h of hazardsStable as any[]) {
    const st = safeStatus(h);
    if (st === "resolved") resolved++;
    else active++;
    if (h.severity === 3) high++;
    else if (h.severity === 2) med++;
    else low++;
  }
  return { high, med, low, active, resolved };
}, [hazardsStable]);
const topBar = (
  <View
    style={{
      position: "absolute",
      top: 12,

```

```

left: 12,
right: 12,
padding: 10,
borderRadius: 14,
backgroundColor: "rgba(2,6,23,0.74)",
borderWidth: 1,
borderColor: "rgba(255,255,255,0.12)",
}}
>
<View style={{ flexDirection: "row", alignItems: "center", justifyContent: "space-between" }}>
<Text style={{ color: "white", fontWeight: "900", letterSpacing: 0.4 }}>
LGU HAZARD OPS
</Text>
<Pressable
onPress={() => setShowResolved((v) => !v)}
style={{
paddingVertical: 8,
paddingHorizontal: 12,
borderRadius: 999,
backgroundColor: showResolved ? "rgba(148,163,184,0.22)" : "rgba(239,68,68,0.18)",
borderWidth: 1,
borderColor: "rgba(255,255,255,0.15)",
}}
>
<Text style={{ color: "white", fontWeight: "900", fontSize: 12 }}>
{showResolved ? "Resolved: ON" : "Resolved: OFF"}
</Text>
</Pressable>
</View>
<View style={{ flexDirection: "row", gap: 8, marginTop: 10 }}>
<View style={{ flex: 1, borderRadius: 12, padding: 10, backgroundColor: "rgba(239,68,68,0.16)",
borderWidth: 1, borderColor: "rgba(239,68,68,0.22)" }}>
<Text style={{ color: "white", fontWeight: "900", fontSize: 12 }}>HIGH</Text>
<Text style={{ color: "white", fontWeight: "900", fontSize: 18 }}>{counts.high}</Text>
</View>
<View style={{ flex: 1, borderRadius: 12, padding: 10, backgroundColor: "rgba(245,158,11,0.14)",
borderWidth: 1, borderColor: "rgba(245,158,11,0.20)" }}>
<Text style={{ color: "white", fontWeight: "900", fontSize: 12 }}>MED</Text>
<Text style={{ color: "white", fontWeight: "900", fontSize: 18 }}>{counts.med}</Text>
</View>
<View style={{ flex: 1, borderRadius: 12, padding: 10, backgroundColor: "rgba(16,185,129,0.14)",
borderWidth: 1, borderColor: "rgba(16,185,129,0.20)" }}>
<Text style={{ color: "white", fontWeight: "900", fontSize: 12 }}>LOW</Text>
<Text style={{ color: "white", fontWeight: "900", fontSize: 18 }}>{counts.low}</Text>
</View>
</View>
<View style={{ flexDirection: "row", gap: 8, marginTop: 10 }}>
<Text style={{ color: "rgba(226,232,240,0.85)", fontWeight: "800", fontSize: 12 }}>
Active: {counts.active} • Resolved: {counts.resolved}
</Text>
<Pressable
onPress={safeReloadHazards}
style={{
marginLeft: "auto",
paddingVertical: 6,
paddingHorizontal: 10,
borderRadius: 999,
backgroundColor: "rgba(37,99,235,0.18)",
borderWidth: 1,
borderColor: "rgba(37,99,235,0.22)",
}}
disabled={isMutating}
>
<Text style={{ color: "white", fontWeight: "900", fontSize: 12 }}>
REFRESH
</Text>
</Pressable>
</View>
</View>

```

```

);
const bottomReport = (
<View style={{ position: "absolute", left: 14, right: 14, bottom: 88 }}>
<Pressable
onPress={enterPlacingMode}
style={{
backgroundColor: "#2563eb",
paddingVertical: 16,
borderRadius: 18,
alignItems: "center",
shadowColor: "#000",
shadowOpacity: 0.25,
shadowRadius: 10,
shadowOffset: { width: 0, height: 6 },
elevation: 6,
opacity: isMutating ? 0.7 : 1,
}}
disabled={isMutating}
>
<Text style={{ color: "white", fontWeight: "900", fontSize: 16, letterSpacing: 0.3 }}>
+ REPORT HAZARD
</Text>
<Text style={{ color: "rgba(226,232,240,0.85)", fontWeight: "800", fontSize: 12, marginTop: 6 }}>
Long-press also works (primary)
</Text>
</Pressable>
</View>
);
const recenterBtn = (
<Pressable
onPress={recenter}
style={{
position: "absolute",
right: 14,
bottom: 166,
paddingVertical: 10,
paddingHorizontal: 12,
borderRadius: 18,
backgroundColor: "rgba(0,0,0,0.85)",
borderWidth: 1,
borderColor: "rgba(255,255,255,0.22)",
flexDirection: "row",
gap: 10,
alignItems: "center",
justifyContent: "center",
shadowColor: "#000",
shadowOpacity: 0.30,
shadowRadius: 12,
shadowOffset: { width: 0, height: 8 },
elevation: 9,
opacity: userCoord ? 1 : 0.45,
}}
disabled={!userCoord}
>
<View
style={{
width: 42,
height: 42,
borderRadius: 21,
backgroundColor: "rgba(37,99,235,0.95)",
alignItems: "center",
justifyContent: "center",
}}
>
<Text style={{ color: "white", fontSize: 22, fontWeight: "900" }}>■</Text>
</View>
<View style={{ alignItems: "flex-start" }}>
<Text style={{ color: "white", fontWeight: "900", fontSize: 12, letterSpacing: 0.2 }}>
RECENTER

```

```

</Text>
<Text style={{ color: "rgba(226,232,240,0.85)", fontWeight: "800", fontSize: 11 }}>
My location
</Text>
</View>
</Pressable>
);
const placingOverlay = placementMode ? (
<>
{ /* Center pin */}
<View
pointerEvents="none"
style={{
position: "absolute",
left: 0,
right: 0,
top: 0,
bottom: 0,
alignItems: "center",
justifyContent: "center",
}}
>
<View
style={{
width: 18,
height: 18,
borderRadius: 9,
backgroundColor: "#dc2626",
borderWidth: 3,
borderColor: "white",
shadowColor: "#000",
shadowOpacity: 0.25,
shadowRadius: 8,
shadowOffset: { width: 0, height: 4 },
elevation: 6,
}}
/>
<View
style={{
marginTop: 10,
paddingHorizontal: 12,
paddingVertical: 8,
borderRadius: 999,
backgroundColor: "rgba(2,6,23,0.75)",
}}
>
<Text style={{ color: "white", fontWeight: "900", fontSize: 12 }}>
Move map • Confirm at center pin
</Text>
</View>
</View>
{ /* Panel */}
<View
style={{
position: "absolute",
left: 12,
right: 12,
bottom: 88,
backgroundColor: "rgba(2,6,23,0.92)",
borderRadius: 20,
padding: 14,
borderWidth: 1,
borderColor: "rgba(255,255,255,0.12)",
}}
>
<Text style={{ color: "white", fontWeight: "900", fontSize: 14, marginBottom: 10 }}>
Report a hazard (LGU)
</Text>
{ /* Severity */}

```

```

<View style={{ flexDirection: "row", gap: 10, marginBottom: 10 }}>
{SEVERITIES.map((s) => (
<Pressable
key={s.key}
onPress={() => setSeverity(s.key)}
style={{
flex: 1,
paddingVertical: 12,
borderRadius: 14,
alignItems: "center",
backgroundColor: severity === s.key ? s.bg : "rgba(148,163,184,0.20)",
borderWidth: 1,
borderColor: "rgba(255,255,255,0.10)",
}}
>
<Text style={{ color: "white", fontWeight: "900" }}>{s.label}</Text>
</Pressable>
))}
</View>
{/* Type */}
<View style={{ flexDirection: "row", flexWrap: "wrap", gap: 8, marginBottom: 12 }}>
{HAZARD_TYPES.map((t) => {
const active = type === t.key;
return (
<Pressable
key={t.key}
onPress={() => setType(t.key)}
style={{
paddingVertical: 10,
paddingHorizontal: 12,
borderRadius: 999,
backgroundColor: active ? "rgba(37,99,235,0.85)" : "rgba(148,163,184,0.18)",
borderWidth: 1,
borderColor: "rgba(255,255,255,0.10)",
}}
>
<Text style={{ color: "white", fontWeight: "800", fontSize: 12 }}>
{t.label}
</Text>
</Pressable>
);
)}}
</View>
<View style={{ flexDirection: "row", gap: 10 }}>
<Pressable
onPress={cancelPlacement}
style={{
flex: 1,
backgroundColor: "rgba(148,163,184,0.22)",
paddingVertical: 14,
borderRadius: 14,
alignItems: "center",
}}
disabled={isMutating}
>
<Text style={{ color: "white", fontWeight: "900" }}>CANCEL</Text>
</Pressable>
<Pressable
onPress={confirmPlacement}
style={{
flex: 1,
backgroundColor: "#16a34a",
paddingVertical: 14,
borderRadius: 14,
alignItems: "center",
opacity: isMutating ? 0.7 : 1,
}}
disabled={isMutating}
>

```

```

<Text style={{ color: "white", fontWeight: "900" }}>
{isMutating ? "SAVING..." : "CONFIRM"}
</Text>
</Pressable>
</View>
</View>
</>
) : null;
const selectedCard = selected ? (
<View
style={{
position: "absolute",
left: 12,
right: 12,
bottom: 88,
backgroundColor: "rgba(2,6,23,0.92)",
borderRadius: 20,
padding: 14,
borderWidth: 1,
borderColor: "rgba(255,255,255,0.12)",
}}
>
<View style={{ flexDirection: "row", justifyContent: "space-between", alignItems: "center" }}>
<Text style={{ color: "white", fontWeight: "900", fontSize: 14 }}>
{shortTypeLabel((selected as any).type)} • {safeStatus(selected) === "resolved" ? "RESOLVED" :
"ACTIVE"}
</Text>
<View
style={{
paddingVertical: 6,
paddingHorizontal: 10,
borderRadius: 999,
backgroundColor: safeStatus(selected) === "resolved" ? "rgba(148,163,184,0.18)" :
"rgba(16,185,129,0.14)",
borderWidth: 1,
borderColor: "rgba(255,255,255,0.12)",
}}
>
<Text style={{ color: "white", fontWeight: "900", fontSize: 12 }}>
{safeStatus(selected) === "resolved" ? "HISTORICAL" : "ACTION"}
</Text>
</View>
</View>
<Text style={{ color: "rgba(226,232,240,0.85)", marginTop: 8, fontStyle: "italic" }}>
{safeStatus(selected) === "resolved"
? "Resolved hazards are kept for historical accountability."
: "Resolve after validation; delete only if duplicate/invalid."}
</Text>
<View style={{ flexDirection: "row", gap: 10, marginTop: 12 }}>
<Pressable
onPress={() => setSelectedId(null)}
style={{
flex: 1,
backgroundColor: "rgba(148,163,184,0.20)",
paddingVertical: 12,
borderRadius: 14,
alignItems: "center",
}}
disabled={isMutating}
>
<Text style={{ color: "white", fontWeight: "900" }}>CLOSE</Text>
</Pressable>
{safeStatus(selected) !== "resolved" ? (
<Pressable
onPress={resolveSelected}
style={{
flex: 1,
backgroundColor: "#16a34a",
paddingVertical: 12,

```

```

borderRadius: 14,
alignItems: "center",
opacity: isMutating ? 0.7 : 1,
}}
disabled={isMutating}
>
<Text style={{ color: "white", fontWeight: "900" }}>
{isMutating ? "..." : "RESOLVE"}
</Text>
</Pressable>
) : (
<View style={{ flex: 1 }} />
)}
<Pressable
onPress={deleteSelected}
style={{
flex: 1,
backgroundColor: "#991b1b",
paddingVertical: 12,
borderRadius: 14,
alignItems: "center",
opacity: isMutating ? 0.7 : 1,
}}
disabled={isMutating}
>
<Text style={{ color: "white", fontWeight: "900" }}>
{isMutating ? "..." : "DELETE"}
</Text>
</Pressable>
</View>
</View>
) : null;
return (
<View style={{ flex: 1 }}>
<MapView onRegionChangeComplete={setRegion}
ref={(r) => (mapRef.current = r)}
style={{ flex: 1 }}
initialRegion={region}
onLongPress={(e: MapPressEvent) => {
// LOCKED: long-press primary trigger
if (!isMutating) enterPlacingMode();
}}
showsUserLocation={true}
showsMyLocationButton={false}
>
{visibleHazards.map((h: any) => {
const st = safeStatus(h);
const pinColor = st === "resolved" ? "#64748b" : severityColor(h.severity);
const opacity = st === "resolved" ? 0.25 : 1.0;
// Remount marker when status changes to avoid reuse bleed
if (!hasValidCoord(h)) return null;
const markerKey = `${h.id}-${st}`;
return (
<Marker
key={markerKey}
coordinate={{ latitude: h.latitude, longitude: h.longitude }}
pinColor={pinColor}
opacity={opacity}
tracksViewChanges={false}
onPress={() => onMarkerPress(String(h.id))}
/>
);
}})
</MapView>
{topBar}
{recenterBtn}
{placementMode ? placingOverlay : selected ? selectedCard : bottomReport}
</View>
);

```

```
}
```

lib/hazards.ts

```
import { supabase } from "../supabase";
import { v4 as uuidv4 } from "uuid";

export type HazardStatus = "reported" | "resolved";

export type Hazard = {
  id: string;
  latitude: number;
  longitude: number;
  type: string;
  severity: number; // 1|2|3
  status: HazardStatus;
  created_at?: string | null;
};

function requireSupabase() {
  // In Expo Go / dev, missing env happens. Don't freeze splash-return null and let UI load.
  return supabase;
}

export async function fetchHazards(): Promise<Hazard[]> {
  const sb = requireSupabase();
  if (!sb) return [];

  const { data, error } = await sb
    .from("hazards")
    .select(".*")
    .order("created_at", { ascending: false });

  if (error) throw error;
  return (data ?? []) as Hazard[];
}

export async function createHazard(input: {
  latitude: number;
  longitude: number;
  type: string;
  severity: number;
}): Promise<Hazard> {
  const sb = requireSupabase();
  const hazard: Hazard = {
    id: uuidv4(),
    latitude: input.latitude,
    longitude: input.longitude,
    type: input.type,
    severity: input.severity,
    status: "reported",
  };

  if (!sb) return hazard;

  const { error } = await sb.from("hazards").insert({
    id: hazard.id,
    latitude: hazard.latitude,
    longitude: hazard.longitude,
    type: hazard.type,
    severity: hazard.severity,
    status: hazard.status,
  });

  if (error) throw error;
  return hazard;
}
```

```

export async function resolveHazard(id: string) {
  const sb = requireSupabase();
  if (!sb) return;

  const { error } = await sb.from("hazards").update({ status: "resolved" }).eq("id", id);
  if (error) throw error;
}

export async function deleteHazard(id: string) {
  const sb = requireSupabase();
  if (!sb) return;

  const { error } = await sb.from("hazards").delete().eq("id", id);
  if (error) throw error;
}

```

.lock_hashes.txt

```

34104bb71266f6f19f2b45fa13b0257a69c8abac2b1a251cbfd59077b8280953  lib/hazards.ts
fab661c3668acd1ce9536f965f55b9c7714902988ab60d52be3f4a51a11572c2  app/lgu/map.base.tsx

```

.lock_hazards_ts.txt

```

1  import { supabase } from "../supabase";
2  import { v4 as uuidv4 } from "uuid";
3
4  export type HazardStatus = "reported" | "resolved";
5
6  export type Hazard = {
7    id: string;
8    latitude: number;
9    longitude: number;
10   type: string;
11   severity: 1 | 2 | 3; // UI uses 1|2|3
12   status: HazardStatus;
13   created_at?: string | null;
14 };
15
16 type DbSeverity = "LOW" | "MEDIUM" | "HIGH";
17
18 function requireSupabase() {
19   return supabase;
20 }
21
22 function toDbSeverity(sev: number): DbSeverity {
23   if (sev === 3) return "HIGH";
24   if (sev === 2) return "MEDIUM";
25   return "LOW";
26 }
27
28 function fromDbSeverity(sev: any): 1 | 2 | 3 {
29   if (sev === "HIGH") return 3;
30   if (sev === "MEDIUM") return 2;
31   return 1; // LOW (or fallback)
32 }
33
34 export async function fetchHazards(): Promise<Hazard[]> {
35   const sb = requireSupabase();
36   if (!sb) return [];
37
38   const { data, error } = await sb
39     .from("hazards")
40     .select("*")
41     .order("created_at", { ascending: false });
42
43   if (error) throw error;
44

```

```

45   const rows = (data ?? []) as any[];
46   return rows.map((h) => ({
47     id: String(h.id),
48     latitude: Number(h.latitude),
49     longitude: Number(h.longitude),
50     type: String(h.type ?? "pothole"),
51     severity: fromDbSeverity(h.severity),
52     status: (h?.status === "resolved" ? "resolved" : "reported") as HazardStatus,
53     created_at: h.created_at ?? null,
54   }));
55 }
56
57 export async function createHazard(input: {
58   latitude: number;
59   longitude: number;
60   type: string;
61   severity: 1 | 2 | 3;
62 }): Promise<Hazard> {
63   const sb = requireSupabase();
64
65   const hazard: Hazard = {
66     id: uuidv4(),
67     latitude: input.latitude,
68     longitude: input.longitude,
69     type: input.type,
70     severity: input.severity,
71     status: "reported",
72   };
73
74   const payload = {
75     id: hazard.id,
76     latitude: hazard.latitude,
77     longitude: hazard.longitude,
78     type: hazard.type,
79     severity: toDbSeverity(hazard.severity), // ■ DB expects LOW|MEDIUM|HIGH
80     status: hazard.status,
81     source: "mobile",
82   };
83
84   console.log("[FINDD] inserting hazard", payload);
85
86   if (!sb) return hazard;
87
88   const { error } = await sb.from("hazards").insert(payload);
89
90   if (error) {
91     console.warn("[FINDD] insert failed", error);
92     throw error;
93   }
94
95   return hazard;
96 }
97
98 export async function resolveHazard(id: string) {
99   const sb = requireSupabase();
100   if (!sb) return;
101
102   const { error } = await sb.from("hazards").update({ status: "resolved" }).eq("id", id);
103   if (error) throw error;
104 }
105
106 export async function deleteHazard(id: string) {
107   const sb = requireSupabase();
108   if (!sb) return;
109
110   const { error } = await sb.from("hazards").delete().eq("id", id);
111   if (error) throw error;
112 }

```

.lock_map_base_tsx.legend_v1.txt

```

import React, { useEffect, useMemo, useRef, useState } from "react";
import { View, Text, Pressable, Dimensions, Platform } from "react-native";
import MapView, { Marker, Region, MapPressEvent } from "react-native-maps";
import * as Location from "expo-location";
import {
  fetchHazards,
  createHazard,
  resolveHazard,
  deleteHazard,
  Hazard,
} from "@lib/hazards";

/* ----- LGU OPS CONFIG ----- */

const DEFAULT_REGION: Region = {
  latitude: 14.5995,
  longitude: 120.9842,
  latitudeDelta: 0.08,
  longitudeDelta: 0.08,
};

const SEVERITIES = [
  { key: 3, label: "HIGH", bg: "#ef4444" },
  { key: 2, label: "MED", bg: "#f59e0b" },
  { key: 1, label: "LOW", bg: "#10b981" },
] as const;

const HAZARD_TYPES = [
  { key: "pothole", label: "Pothole" },
  { key: "flood", label: "Flood" },
  { key: "crack", label: "Crack" },
  { key: "debris", label: "Debris" },
  { key: "construction", label: "Construction" },
] as const;

type HazardType = (typeof HAZARD_TYPES)[number]["key"];

function severityColor(sev: number) {
  if (sev === 3) return "#ef4444";
  if (sev === 2) return "#f59e0b";
  return "#10b981";
}

function safeStatus(h: any): "reported" | "resolved" {
  return h?.status === "resolved" ? "resolved" : "reported";
}

function shortTypeLabel(t: string) {
  const found = HAZARD_TYPES.find((x) => x.key === t);
  return found ? found.label : String(t || "Hazard");
}

// ----- LGU MARKER VISUAL SYSTEM (LOCKED) -----
function typeGlyph(type) {
  switch (type) {
    case "pothole": return "PH";
    case "flood": return "FL";
    case "crack": return "CR";
    case "debris": return "DB";
    case "construction": return "CN";
    default: return "HZ";
  }
}

function getMarkerStyle(sev, status) {
  const r = status === "resolved";
  if (sev === 3) return { size: r ? 26 : 34, radius: 6, border: 4, opacity: r ? 0.35 : 1, elevation: r ? 1 : 9 };
  if (sev === 2) return { size: r ? 22 : 28, radius: 8, border: 3, opacity: r ? 0.40 : 1, elevation:

```

```

r ? 1 : 7 };
return { size: r ? 18 : 22, radius: 999, border: 3, opacity: r ? 0.45 : 1, elevation: r ? 1 : 5 };
}

/* ----- COMPONENT ----- */

export default function LGUMap() {
  const mapRef = useRef<MapView | null>(null);

  const [region, setRegion] = useState<Region>(DEFAULT_REGION);
  const [userCoord, setUserCoord] =
    useState<{ latitude: number; longitude: number } | null>(null);

  const [hazards, setHazards] = useState<Hazard[]>([]);
  const [selectedId, setSelectedId] = useState<string | null>(null);
  const [isMutating, setIsMutating] = useState(false);

  // Center-pin placement mode (LOCKED)
  const [placementMode, setPlacementMode] = useState(false);
  const [severity, setSeverity] = useState<1 | 2 | 3>(2);
  const [type, setType] = useState<HazardType>("pothole");

  // LGU filters
  const [severityFilter, setSeverityFilter] = useState<Record<number, boolean>>({
    3: true,
    2: true,
    1: true,
  });
  const [typeFilter, setTypeFilter] = useState<Record<HazardType, boolean>>({
    pothole: true,
    flood: true,
    crack: true,
    debris: true,
    construction: true,
  });
  const [showResolved, setShowResolved] = useState(true);

  async function safeReloadHazards() {
    try {
      const next = await fetchHazards();
      setHazards(Array.isArray(next) ? next : []);
    } catch (e) {
      console.warn("[LGUMap] fetchHazards failed", e);
      setHazards([]);
    }
  }

  // Location bootstrap + initial load
  useEffect(() => {
    (async () => {
      try {
        const { status } = await Location.requestForegroundPermissionsAsync();
        if (status === "granted") {
          const loc = await Location.getLastKnownPositionAsync({});
          if (loc?.coords) {
            const r: Region = {
              latitude: loc.coords.latitude,
              longitude: loc.coords.longitude,
              latitudeDelta: 0.06,
              longitudeDelta: 0.06,
            };
            setUserCoord({ latitude: r.latitude, longitude: r.longitude });
            setRegion(r);
          }
        }
      } catch {
        // keep default region
      }
      await safeReloadHazards();
    })();
  }, []);
}

```

```

    })();
  }, []);

const selected = useMemo(() => {
  if (!selectedId) return null;
  return hazards.find((h: any) => String(h.id) === String(selectedId)) ?? null;
}, [selectedId, hazards]);

// Clear selection if it disappears
useEffect(() => {
  if (selectedId && !hazards.some((h: any) => String(h.id) === String(selectedId))) {
    setSelectedId(null);
  }
}, [hazards, selectedId]);

// LOCKED: single source of truth
function enterPlacingMode() {
  setSelectedId(null);
  setPlacementMode(true);
}

function cancelPlacement() {
  setPlacementMode(false);
}

async function confirmPlacement() {
  if (!mapRef.current || isMutating) return;

  try {
    setIsMutating(true);

    const { width, height } = Dimensions.get("window");
    // Best-effort: center of the screen. Good enough for ops.
    const coord = await mapRef.current.coordinateForPoint({
      x: width / 2,
      y: height / 2,
    });

    await createHazard({
      latitude: coord.latitude,
      longitude: coord.longitude,
      severity,
      type,
    });

    await safeReloadHazards();
  } catch (e) {
    console.warn("[LGUMap] confirmPlacement failed", e);
  } finally {
    setIsMutating(false);
    setPlacementMode(false);
  }
}

function onMarkerPress(id: string) {
  if (isMutating || placementMode) return;
  setSelectedId(id);
}

async function resolveSelected() {
  if (!selected || isMutating) return;
  try {
    setIsMutating(true);
    await resolveHazard(String((selected as any).id));
    await safeReloadHazards();
  } catch (e) {
    console.warn("[LGUMap] resolveSelected failed", e);
  } finally {
    setIsMutating(false);
  }
}

```

```

    }
  }

  async function deleteSelected() {
    if (!selected || isMutating) return;

    const id = String((selected as any).id);
    setSelectedId(null);

    try {
      setIsMutating(true);
      await deleteHazard(id);
      await safeReloadHazards();
    } catch (e) {
      console.warn("[LGUMap] deleteSelected failed", e);
      await safeReloadHazards();
    } finally {
      setIsMutating(false);
    }
  }

  function recenter() {
    if (!mapRef.current || !userCoord) return;

    const r: Region = {
      latitude: userCoord.latitude,
      longitude: userCoord.longitude,
      latitudeDelta: 0.06,
      longitudeDelta: 0.06,
    };

    setRegion(r);
    mapRef.current.animateToRegion(r, 350);
  }

  const hazardsStable = useMemo(() => {
    const arr = [...hazards] as any[];
    arr.sort((a, b) => String(a.id).localeCompare(String(b.id)));
    return arr;
  }, [hazards]);

  const visibleHazards = useMemo(() => {
    return hazardsStable.filter((h: any) => {
      const st = safeStatus(h);
      if (!showResolved && st === "resolved") return false;
      if (!severityFilter[h.severity]) return false;
      if (!typeFilter[h.type as HazardType]) return false;
      return true;
    });
  }, [hazardsStable, severityFilter, typeFilter, showResolved]);

  const counts = useMemo(() => {
    let high = 0, med = 0, low = 0;
    let active = 0, resolved = 0;

    for (const h of hazardsStable as any[]) {
      const st = safeStatus(h);
      if (st === "resolved") resolved++;
      else active++;

      if (h.severity === 3) high++;
      else if (h.severity === 2) med++;
      else low++;
    }
    return { high, med, low, active, resolved };
  }, [hazardsStable]);

  const topBar = (
    <View

```

```

style={{
  position: "absolute",
  top: 12,
  left: 12,
  right: 12,
  padding: 10,
  borderRadius: 14,
  backgroundColor: "rgba(2,6,23,0.74)",
  borderWidth: 1,
  borderColor: "rgba(255,255,255,0.12)",
}}
>
<View style={{ flexDirection: "row", alignItems: "center", justifyContent: "space-between" }}>
  <Text style={{ color: "white", fontWeight: "900", letterSpacing: 0.4 }}>
    LGU HAZARD OPS
  </Text>

  <Pressable
    onPress={() => setShowResolved((v) => !v)}
    style={{
      paddingVertical: 8,
      paddingHorizontal: 12,
      borderRadius: 999,
      backgroundColor: showResolved ? "rgba(148,163,184,0.22)" : "rgba(239,68,68,0.18)",
      borderWidth: 1,
      borderColor: "rgba(255,255,255,0.15)",
    }}
  >
    <Text style={{ color: "white", fontWeight: "900", fontSize: 12 }}>
      {showResolved ? "Resolved: ON" : "Resolved: OFF"}
    </Text>
  </Pressable>
</View>

<View style={{ flexDirection: "row", gap: 8, marginTop: 10 }}>
  <View style={{ flex: 1, borderRadius: 12, padding: 10, backgroundColor:
"rgba(239,68,68,0.16)", borderWidth: 1, borderColor: "rgba(239,68,68,0.22)" }}>
    <Text style={{ color: "white", fontWeight: "900", fontSize: 12 }}>HIGH</Text>
    <Text style={{ color: "white", fontWeight: "900", fontSize: 18 }}>{counts.high}</Text>
  </View>
  <View style={{ flex: 1, borderRadius: 12, padding: 10, backgroundColor:
"rgba(245,158,11,0.14)", borderWidth: 1, borderColor: "rgba(245,158,11,0.20)" }}>
    <Text style={{ color: "white", fontWeight: "900", fontSize: 12 }}>MED</Text>
    <Text style={{ color: "white", fontWeight: "900", fontSize: 18 }}>{counts.med}</Text>
  </View>
  <View style={{ flex: 1, borderRadius: 12, padding: 10, backgroundColor:
"rgba(16,185,129,0.14)", borderWidth: 1, borderColor: "rgba(16,185,129,0.20)" }}>
    <Text style={{ color: "white", fontWeight: "900", fontSize: 12 }}>LOW</Text>
    <Text style={{ color: "white", fontWeight: "900", fontSize: 18 }}>{counts.low}</Text>
  </View>
</View>

<View style={{ flexDirection: "row", gap: 8, marginTop: 10 }}>
  <Text style={{ color: "rgba(226,232,240,0.85)", fontWeight: "800", fontSize: 12 }}>
    Active: {counts.active} • Resolved: {counts.resolved}
  </Text>

  <Pressable
    onPress={safeReloadHazards}
    style={{
      marginLeft: "auto",
      paddingVertical: 6,
      paddingHorizontal: 10,
      borderRadius: 999,
      backgroundColor: "rgba(37,99,235,0.18)",
      borderWidth: 1,
      borderColor: "rgba(37,99,235,0.22)",
    }}
    disabled={isMutating}
  >

```

```

    >
    <Text style={{ color: "white", fontWeight: "900", fontSize: 12 }}>
      REFRESH
    </Text>
  </Pressable>
</View>
</View>
);

const bottomReport = (
  <View style={{ position: "absolute", left: 14, right: 14, bottom: 88 }}>
    <Pressable
      onPress={enterPlacingMode}
      style={{
        backgroundColor: "#2563eb",
        paddingVertical: 16,
        borderRadius: 18,
        alignItems: "center",
        shadowColor: "#000",
        shadowOpacity: 0.25,
        shadowRadius: 10,
        shadowOffset: { width: 0, height: 6 },
        elevation: 6,
        opacity: isMutating ? 0.7 : 1,
      }}
      disabled={isMutating}
    >
      <Text style={{ color: "white", fontWeight: "900", fontSize: 16, letterSpacing: 0.3 }}>
        + REPORT HAZARD
      </Text>
      <Text style={{ color: "rgba(226,232,240,0.85)", fontWeight: "800", fontSize: 12, marginTop:
6 }}>
        Long-press also works (primary)
      </Text>
    </Pressable>
  </View>
);

const recenterBtn = (
  <Pressable
    onPress={recenter}
    style={{
      position: "absolute",
      right: 14,
      bottom: 166,
      paddingVertical: 10,
      paddingHorizontal: 12,
      borderRadius: 18,
      backgroundColor: "rgba(0,0,0,0.85)",
      borderWidth: 1,
      borderColor: "rgba(255,255,255,0.22)",
      flexDirection: "row",
      gap: 10,
      alignItems: "center",
      justifyContent: "center",
      shadowColor: "#000",
      shadowOpacity: 0.30,
      shadowRadius: 12,
      shadowOffset: { width: 0, height: 8 },
      elevation: 9,
      opacity: userCoord ? 1 : 0.45,
    }}
    disabled={!userCoord}
  >
    <View
      style={{
        width: 42,
        height: 42,
        borderRadius: 21,

```

```

        backgroundColor: "rgba(37,99,235,0.95)",
        alignItems: "center",
        justifyContent: "center",
    }}
    >
    <Text style={{ color: "white", fontSize: 22, fontWeight: "900" }}>■</Text>
</View>

<View style={{ alignItems: "flex-start" }}>
    <Text style={{ color: "white", fontWeight: "900", fontSize: 12, letterSpacing: 0.2 }}>
        RECENTER
    </Text>
    <Text style={{ color: "rgba(226,232,240,0.85)", fontWeight: "800", fontSize: 11 }}>
        My location
    </Text>
</View>
</Pressable>
);

const placingOverlay = placementMode ? (
    <>
        { /* Center pin */ }
        <View
            pointerEvents="none"
            style={{
                position: "absolute",
                left: 0,
                right: 0,
                top: 0,
                bottom: 0,
                alignItems: "center",
                justifyContent: "center",
            }}
        >
            <View
                style={{
                    width: 18,
                    height: 18,
                    borderRadius: 9,
                    backgroundColor: "#dc2626",
                    borderWidth: 3,
                    borderColor: "white",
                    shadowColor: "#000",
                    shadowOpacity: 0.25,
                    shadowRadius: 8,
                    shadowOffset: { width: 0, height: 4 },
                    elevation: 6,
                }}
            />
            <View
                style={{
                    marginTop: 10,
                    paddingHorizontal: 12,
                    paddingVertical: 8,
                    borderRadius: 999,
                    backgroundColor: "rgba(2,6,23,0.75)",
                }}
            >
                <Text style={{ color: "white", fontWeight: "900", fontSize: 12 }}>
                    Move map • Confirm at center pin
                </Text>
            </View>
        </View>

        { /* Panel */ }
        <View
            style={{
                position: "absolute",
                left: 12,

```

```

        right: 12,
        bottom: 88,
        backgroundColor: "rgba(2,6,23,0.92)",
        borderRadius: 20,
        padding: 14,
        borderWidth: 1,
        borderColor: "rgba(255,255,255,0.12)",
    }}
>
<Text style={{ color: "white", fontWeight: "900", fontSize: 14, marginBottom: 10 }}>
    Report a hazard (LGU)
</Text>

{/* Severity */}
<View style={{ flexDirection: "row", gap: 10, marginBottom: 10 }}>
    {SEVERITIES.map((s) => (
        <Pressable
            key={s.key}
            onPress={() => setSeverity(s.key)}
            style={{
                flex: 1,
                paddingVertical: 12,
                borderRadius: 14,
                alignItems: "center",
                backgroundColor: severity === s.key ? s.bg : "rgba(148,163,184,0.20)",
                borderWidth: 1,
                borderColor: "rgba(255,255,255,0.10)",
            }}
        >
            <Text style={{ color: "white", fontWeight: "900" }}>{s.label}</Text>
        </Pressable>
    ))}
</View>

{/* Type */}
<View style={{ flexDirection: "row", flexWrap: "wrap", gap: 8, marginBottom: 12 }}>
    {HAZARD_TYPES.map((t) => {
        const active = type === t.key;
        return (
            <Pressable
                key={t.key}
                onPress={() => setType(t.key)}
                style={{
                    paddingVertical: 10,
                    paddingHorizontal: 12,
                    borderRadius: 999,
                    backgroundColor: active ? "rgba(37,99,235,0.85)" : "rgba(148,163,184,0.18)",
                    borderWidth: 1,
                    borderColor: "rgba(255,255,255,0.10)",
                }}
            >
                <Text style={{ color: "white", fontWeight: "800", fontSize: 12 }}>
                    {t.label}
                </Text>
            </Pressable>
        );
    })}
</View>

<View style={{ flexDirection: "row", gap: 10 }}>
    <Pressable
        onPress={cancelPlacement}
        style={{
            flex: 1,
            backgroundColor: "rgba(148,163,184,0.22)",
            paddingVertical: 14,
            borderRadius: 14,
            alignItems: "center",
        }}
    >

```

```

        disabled={isMutating}
      >
      <Text style={{ color: "white", fontWeight: "900" }}>CANCEL</Text>
    </Pressable>

    <Pressable
      onPress={confirmPlacement}
      style={{
        flex: 1,
        backgroundColor: "#16a34a",
        paddingVertical: 14,
        borderRadius: 14,
        alignItems: "center",
        opacity: isMutating ? 0.7 : 1,
      }}
      disabled={isMutating}
    >
      <Text style={{ color: "white", fontWeight: "900" }}>
        {isMutating ? "SAVING..." : "CONFIRM"}
      </Text>
    </Pressable>
  </View>
</View>
</>
) : null;

const selectedCard = selected ? (
  <View
    style={{
      position: "absolute",
      left: 12,
      right: 12,
      bottom: 88,
      backgroundColor: "rgba(2,6,23,0.92)",
      borderRadius: 20,
      padding: 14,
      borderWidth: 1,
      borderColor: "rgba(255,255,255,0.12)",
    }}
  >
    <View style={{ flexDirection: "row", justifyContent: "space-between", alignItems: "center" }}>
      <Text style={{ color: "white", fontWeight: "900", fontSize: 14 }}>
        {shortTypeLabel((selected as any).type)} • {safeStatus(selected) === "resolved" ?
"RESOLVED" : "ACTIVE"}
      </Text>

      <View
        style={{
          paddingVertical: 6,
          paddingHorizontal: 10,
          borderRadius: 999,
          backgroundColor: safeStatus(selected) === "resolved" ? "rgba(148,163,184,0.18)" :
"rgba(16,185,129,0.14)",
          borderWidth: 1,
          borderColor: "rgba(255,255,255,0.12)",
        }}
      >
        <Text style={{ color: "white", fontWeight: "900", fontSize: 12 }}>
          {safeStatus(selected) === "resolved" ? "HISTORICAL" : "ACTION"}
        </Text>
      </View>
    </View>

    <Text style={{ color: "rgba(226,232,240,0.85)", marginTop: 8, fontStyle: "italic" }}>
      {safeStatus(selected) === "resolved"
        ? "Resolved hazards are kept for historical accountability."
        : "Resolve after validation; delete only if duplicate/invalid."}
    </Text>

```

```

<View style={{ flexDirection: "row", gap: 10, marginTop: 12 }}>
  <Pressable
    onPress={() => setSelectedId(null)}
    style={{
      flex: 1,
      backgroundColor: "rgba(148,163,184,0.20)",
      paddingVertical: 12,
      borderRadius: 14,
      alignItems: "center",
    }}
    disabled={isMutating}
  >
    <Text style={{ color: "white", fontWeight: "900" }}>CLOSE</Text>
  </Pressable>

  {safeStatus(selected) !== "resolved" ? (
    <Pressable
      onPress={resolveSelected}
      style={{
        flex: 1,
        backgroundColor: "#16a34a",
        paddingVertical: 12,
        borderRadius: 14,
        alignItems: "center",
        opacity: isMutating ? 0.7 : 1,
      }}
      disabled={isMutating}
    >
      <Text style={{ color: "white", fontWeight: "900" }}>
        {isMutating ? "..." : "RESOLVE"}
      </Text>
    </Pressable>
  ) : (
    <View style={{ flex: 1 }} />
  )}

  <Pressable
    onPress={deleteSelected}
    style={{
      flex: 1,
      backgroundColor: "#991b1b",
      paddingVertical: 12,
      borderRadius: 14,
      alignItems: "center",
      opacity: isMutating ? 0.7 : 1,
    }}
    disabled={isMutating}
  >
    <Text style={{ color: "white", fontWeight: "900" }}>
      {isMutating ? "..." : "DELETE"}
    </Text>
  </Pressable>
</View>
</View>
) : null;

return (
  <View style={{ flex: 1 }}>
    <MapView
      ref={(r) => (mapRef.current = r)}
      style={{ flex: 1 }}
      initialRegion={region}
      onLongPress={(e: MapPressEvent) => {
        // LOCKED: long-press primary trigger
        if (!isMutating) enterPlacingMode();
      }}
      showsUserLocation={true}
      showsMyLocationButton={false}
    >

```

```

{visibleHazards.map((h: any) => {
  const st = safeStatus(h);
  const pinColor = st === "resolved" ? "#64748b" : severityColor(h.severity);
  const opacity = st === "resolved" ? 0.25 : 1.0;

  // Remount marker when status changes to avoid reuse bleed
  const markerKey = `${h.id}-${st}`;

  return (
    <Marker
      key={markerKey}
      coordinate={{ latitude: h.latitude, longitude: h.longitude }}
      tracksViewChanges={false}
      onPress={() => onMarkerPress(String(h.id))}
    >
      {(() => {
        const m = getMarkerStyle(h.severity, st);
        return (
          <View
            collapsable={false}
            renderToHardwareTextureAndroid={true}
            style={{
              width: m.size,
              height: m.size,
              borderRadius: m.radius,
              backgroundColor: st === "resolved" ? "#64748b" : severityColor(h.severity),
              borderWidth: m.border,
              borderColor: "white",
              alignItems: "center",
              justifyContent: "center",
              opacity: m.opacity,
              elevation: m.elevation,
            }}
          >
            <Text style={{ color: "white", fontWeight: "900", fontSize: 11 }}>
              {typeGlyph(h.type)}
            </Text>
          </View>
        );
      })()}
    </Marker>
  );
})}
</MapView>

```

```

/* ----- LGU MINI LEGEND (LOCKED) ----- */
<View
  pointerEvents="none"
  style={{
    position: "absolute",
    top: 56,
    right: 12,
    backgroundColor: "rgba(2,6,23,0.85)",
    borderRadius: 12,
    paddingVertical: 6,
    paddingHorizontal: 10,
    flexDirection: "row",
    gap: 8,
    borderWidth: 1,
    borderColor: "rgba(255,255,255,0.12)",
  }}
>
  {[ "PH", "FL", "CR", "DB", "CN" ].map((k) => (
    <Text
      key={k}
      style={{
        color: "white",
        fontWeight: "900",

```

```

        fontSize: 11,
        letterSpacing: 0.5,
      }}
    >
    {k}
  </Text>
))}
</View>

{topBar}
{recenterBtn}
{placementMode ? placingOverlay : selected ? selectedCard : bottomReport}
</View>
);
}

```

.lock_map_base_tsx.marker_visuals.txt

<ENTIRE FILE CONTENT HERE>

.lock_map_base_tsx.marker_visuals_v1.txt

```

import React, { useEffect, useMemo, useRef, useState } from "react";
import { View, Text, Pressable, Dimensions, Platform } from "react-native";
import MapView, { Marker, Region, MapPressEvent } from "react-native-maps";
import * as Location from "expo-location";
import {
  fetchHazards,
  createHazard,
  resolveHazard,
  deleteHazard,
  Hazard,
} from "@lib/hazards";

/* ----- LGU OPS CONFIG ----- */

const DEFAULT_REGION: Region = {
  latitude: 14.5995,
  longitude: 120.9842,
  latitudeDelta: 0.08,
  longitudeDelta: 0.08,
};

const SEVERITIES = [
  { key: 3, label: "HIGH", bg: "#ef4444" },
  { key: 2, label: "MED", bg: "#f59e0b" },
  { key: 1, label: "LOW", bg: "#10b981" },
] as const;

const HAZARD_TYPES = [
  { key: "pothole", label: "Pothole" },
  { key: "flood", label: "Flood" },
  { key: "crack", label: "Crack" },
  { key: "debris", label: "Debris" },
  { key: "construction", label: "Construction" },
] as const;

type HazardType = (typeof HAZARD_TYPES)[number]["key"];

function severityColor(sev: number) {
  if (sev === 3) return "#ef4444";
  if (sev === 2) return "#f59e0b";
  return "#10b981";
}

function safeStatus(h: any): "reported" | "resolved" {

```

```

    return h?.status === "resolved" ? "resolved" : "reported";
}

function shortTypeLabel(t: string) {
    const found = HAZARD_TYPES.find((x) => x.key === t);
    return found ? found.label : String(t || "Hazard");
}

// ----- LGU MARKER VISUAL SYSTEM (LOCKED) -----
function typeGlyph(type) {
    switch (type) {
        case "pothole": return "PH";
        case "flood": return "FL";
        case "crack": return "CR";
        case "debris": return "DB";
        case "construction": return "CN";
        default: return "HZ";
    }
}

function getMarkerStyle(sev, status) {
    const r = status === "resolved";
    if (sev === 3) return { size: r ? 26 : 34, radius: 6, border: 4, opacity: r ? 0.35 : 1, elevation:
r ? 1 : 9 };
    if (sev === 2) return { size: r ? 22 : 28, radius: 8, border: 3, opacity: r ? 0.40 : 1, elevation:
r ? 1 : 7 };
    return { size: r ? 18 : 22, radius: 999, border: 3, opacity: r ? 0.45 : 1, elevation: r ? 1 : 5 };
}

/* ----- COMPONENT ----- */

export default function LGUMap() {
    const mapRef = useRef<MapView | null>(null);

    const [region, setRegion] = useState<Region>(DEFAULT_REGION);
    const [userCoord, setUserCoord] =
        useState<{ latitude: number; longitude: number } | null>(null);

    const [hazards, setHazards] = useState<Hazard[]>([]);
    const [selectedId, setSelectedId] = useState<string | null>(null);
    const [isMutating, setIsMutating] = useState(false);

    // Center-pin placement mode (LOCKED)
    const [placementMode, setPlacementMode] = useState(false);
    const [severity, setSeverity] = useState<1 | 2 | 3>(2);
    const [type, setType] = useState<HazardType>("pothole");

    // LGU filters
    const [severityFilter, setSeverityFilter] = useState<Record<number, boolean>>({
        3: true,
        2: true,
        1: true,
    });
    const [typeFilter, setTypeFilter] = useState<Record<HazardType, boolean>>({
        pothole: true,
        flood: true,
        crack: true,
        debris: true,
        construction: true,
    });
    const [showResolved, setShowResolved] = useState(true);

    async function safeReloadHazards() {
        try {
            const next = await fetchHazards();
            setHazards(Array.isArray(next) ? next : []);
        } catch (e) {
            console.warn("[LGUMap] fetchHazards failed", e);
            setHazards([]);
        }
    }
}

```

```

    }
  }

// Location bootstrap + initial load
useEffect(() => {
  (async () => {
    try {
      const { status } = await Location.requestForegroundPermissionsAsync();
      if (status === "granted") {
        const loc = await Location.getLastKnownPositionAsync({});
        if (loc?.coords) {
          const r: Region = {
            latitude: loc.coords.latitude,
            longitude: loc.coords.longitude,
            latitudeDelta: 0.06,
            longitudeDelta: 0.06,
          };
          setUserCoord({ latitude: r.latitude, longitude: r.longitude });
          setRegion(r);
        }
      }
    } catch {
      // keep default region
    }
    await safeReloadHazards();
  })();
}, []);

const selected = useMemo(() => {
  if (!selectedId) return null;
  return hazards.find((h: any) => String(h.id) === String(selectedId)) ?? null;
}, [selectedId, hazards]);

// Clear selection if it disappears
useEffect(() => {
  if (selectedId && !hazards.some((h: any) => String(h.id) === String(selectedId))) {
    setSelectedId(null);
  }
}, [hazards, selectedId]);

// LOCKED: single source of truth
function enterPlacingMode() {
  setSelectedId(null);
  setPlacementMode(true);
}

function cancelPlacement() {
  setPlacementMode(false);
}

async function confirmPlacement() {
  if (!mapRef.current || isMutating) return;

  try {
    setIsMutating(true);

    const { width, height } = Dimensions.get("window");
    // Best-effort: center of the screen. Good enough for ops.
    const coord = await mapRef.current.coordinateForPoint({
      x: width / 2,
      y: height / 2,
    });

    await createHazard({
      latitude: coord.latitude,
      longitude: coord.longitude,
      severity,
      type,
    });
  }
}

```

```

        await safeReloadHazards();
    } catch (e) {
        console.warn("[LGUMap] confirmPlacement failed", e);
    } finally {
        setIsMutating(false);
        setPlacementMode(false);
    }
}

function onMarkerPress(id: string) {
    if (isMutating || placementMode) return;
    setSelectedId(id);
}

async function resolveSelected() {
    if (!selected || isMutating) return;
    try {
        setIsMutating(true);
        await resolveHazard(String((selected as any).id));
        await safeReloadHazards();
    } catch (e) {
        console.warn("[LGUMap] resolveSelected failed", e);
    } finally {
        setIsMutating(false);
    }
}

async function deleteSelected() {
    if (!selected || isMutating) return;

    const id = String((selected as any).id);
    setSelectedId(null);

    try {
        setIsMutating(true);
        await deleteHazard(id);
        await safeReloadHazards();
    } catch (e) {
        console.warn("[LGUMap] deleteSelected failed", e);
        await safeReloadHazards();
    } finally {
        setIsMutating(false);
    }
}

function recenter() {
    if (!mapRef.current || !userCoord) return;

    const r: Region = {
        latitude: userCoord.latitude,
        longitude: userCoord.longitude,
        latitudeDelta: 0.06,
        longitudeDelta: 0.06,
    };

    setRegion(r);
    mapRef.current.animateToRegion(r, 350);
}

const hazardsStable = useMemo(() => {
    const arr = [...hazards] as any[];
    arr.sort((a, b) => String(a.id).localeCompare(String(b.id)));
    return arr;
}, [hazards]);

const visibleHazards = useMemo(() => {
    return hazardsStable.filter((h: any) => {
        const st = safeStatus(h);

```

```

    if (!showResolved && st === "resolved") return false;
    if (!severityFilter[h.severity]) return false;
    if (!typeFilter[h.type as HazardType]) return false;
    return true;
  });
}, [hazardsStable, severityFilter, typeFilter, showResolved]);

const counts = useMemo(() => {
  let high = 0, med = 0, low = 0;
  let active = 0, resolved = 0;

  for (const h of hazardsStable as any[]) {
    const st = safeStatus(h);
    if (st === "resolved") resolved++;
    else active++;

    if (h.severity === 3) high++;
    else if (h.severity === 2) med++;
    else low++;
  }
  return { high, med, low, active, resolved };
}, [hazardsStable]);

const topBar = (
  <View
    style={{
      position: "absolute",
      top: 12,
      left: 12,
      right: 12,
      padding: 10,
      borderRadius: 14,
      backgroundColor: "rgba(2,6,23,0.74)",
      borderWidth: 1,
      borderColor: "rgba(255,255,255,0.12)",
    }}
  >
    <View style={{ flexDirection: "row", alignItems: "center", justifyContent: "space-between" }}>
      <Text style={{ color: "white", fontWeight: "900", letterSpacing: 0.4 }}>
        LGU HAZARD OPS
      </Text>

      <Pressable
        onPress={() => setShowResolved((v) => !v)}
        style={{
          paddingVertical: 8,
          paddingHorizontal: 12,
          borderRadius: 999,
          backgroundColor: showResolved ? "rgba(148,163,184,0.22)" : "rgba(239,68,68,0.18)",
          borderWidth: 1,
          borderColor: "rgba(255,255,255,0.15)",
        }}
      >
        <Text style={{ color: "white", fontWeight: "900", fontSize: 12 }}>
          {showResolved ? "Resolved: ON" : "Resolved: OFF"}
        </Text>
      </Pressable>
    </View>

    <View style={{ flexDirection: "row", gap: 8, marginTop: 10 }}>
      <View style={{ flex: 1, borderRadius: 12, padding: 10, backgroundColor:
"rgba(239,68,68,0.16)", borderWidth: 1, borderColor: "rgba(239,68,68,0.22)" }}>
        <Text style={{ color: "white", fontWeight: "900", fontSize: 12 }}>HIGH</Text>
        <Text style={{ color: "white", fontWeight: "900", fontSize: 18 }}>{counts.high}</Text>
      </View>
      <View style={{ flex: 1, borderRadius: 12, padding: 10, backgroundColor:
"rgba(245,158,11,0.14)", borderWidth: 1, borderColor: "rgba(245,158,11,0.20)" }}>
        <Text style={{ color: "white", fontWeight: "900", fontSize: 12 }}>MED</Text>
        <Text style={{ color: "white", fontWeight: "900", fontSize: 18 }}>{counts.med}</Text>
      </View>
    </View>
  </View>

```

```

    </View>
    <View style={{ flex: 1, borderRadius: 12, padding: 10, backgroundColor:
"rgba(16,185,129,0.14)", borderWidth: 1, borderColor: "rgba(16,185,129,0.20)" }}>
      <Text style={{ color: "white", fontWeight: "900", fontSize: 12 }}>LOW</Text>
      <Text style={{ color: "white", fontWeight: "900", fontSize: 18 }}>{counts.low}</Text>
    </View>
  </View>

  <View style={{ flexDirection: "row", gap: 8, marginTop: 10 }}>
    <Text style={{ color: "rgba(226,232,240,0.85)", fontWeight: "800", fontSize: 12 }}>
      Active: {counts.active} • Resolved: {counts.resolved}
    </Text>

    <Pressable
      onPress={safeReloadHazards}
      style={{
        marginLeft: "auto",
        paddingVertical: 6,
        paddingHorizontal: 10,
        borderRadius: 999,
        backgroundColor: "rgba(37,99,235,0.18)",
        borderWidth: 1,
        borderColor: "rgba(37,99,235,0.22)",
      }}
      disabled={isMutating}
    >
      <Text style={{ color: "white", fontWeight: "900", fontSize: 12 }}>
        REFRESH
      </Text>
    </Pressable>
  </View>
</View>
);

const bottomReport = (
  <View style={{ position: "absolute", left: 14, right: 14, bottom: 88 }}>
    <Pressable
      onPress={enterPlacingMode}
      style={{
        backgroundColor: "#2563eb",
        paddingVertical: 16,
        borderRadius: 18,
        alignItems: "center",
        shadowColor: "#000",
        shadowOpacity: 0.25,
        shadowRadius: 10,
        shadowOffset: { width: 0, height: 6 },
        elevation: 6,
        opacity: isMutating ? 0.7 : 1,
      }}
      disabled={isMutating}
    >
      <Text style={{ color: "white", fontWeight: "900", fontSize: 16, letterSpacing: 0.3 }}>
        + REPORT HAZARD
      </Text>
      <Text style={{ color: "rgba(226,232,240,0.85)", fontWeight: "800", fontSize: 12, marginTop:
6 }}>
        Long-press also works (primary)
      </Text>
    </Pressable>
  </View>
);

const recenterBtn = (
  <Pressable
    onPress={recenter}
    style={{
      position: "absolute",
      right: 14,

```

```

        bottom: 166,
        paddingVertical: 10,
        paddingHorizontal: 12,
        borderRadius: 18,
        backgroundColor: "rgba(0,0,0,0.85)",
        borderWidth: 1,
        borderColor: "rgba(255,255,255,0.22)",
        flexDirection: "row",
        gap: 10,
        alignItems: "center",
        justifyContent: "center",
        shadowColor: "#000",
        shadowOpacity: 0.30,
        shadowRadius: 12,
        shadowOffset: { width: 0, height: 8 },
        elevation: 9,
        opacity: userCoord ? 1 : 0.45,
    }}
    disabled={!userCoord}
>
<View
  style={{
    width: 42,
    height: 42,
    borderRadius: 21,
    backgroundColor: "rgba(37,99,235,0.95)",
    alignItems: "center",
    justifyContent: "center",
  }}
>
  <Text style={{ color: "white", fontSize: 22, fontWeight: "900" }}>■</Text>
</View>

<View style={{ alignItems: "flex-start" }}>
  <Text style={{ color: "white", fontWeight: "900", fontSize: 12, letterSpacing: 0.2 }}>
    RECENTER
  </Text>
  <Text style={{ color: "rgba(226,232,240,0.85)", fontWeight: "800", fontSize: 11 }}>
    My location
  </Text>
</View>
</Pressable>
);

const placingOverlay = placementMode ? (
  <>
    { /* Center pin */ }
    <View
      pointerEvents="none"
      style={{
        position: "absolute",
        left: 0,
        right: 0,
        top: 0,
        bottom: 0,
        alignItems: "center",
        justifyContent: "center",
      }}
    >
      <View
        style={{
          width: 18,
          height: 18,
          borderRadius: 9,
          backgroundColor: "#dc2626",
          borderWidth: 3,
          borderColor: "white",
          shadowColor: "#000",
          shadowOpacity: 0.25,

```

```

        shadowRadius: 8,
        shadowOffset: { width: 0, height: 4 },
        elevation: 6,
    }}
</>
<View
  style={{
    marginTop: 10,
    paddingHorizontal: 12,
    paddingVertical: 8,
    borderRadius: 999,
    backgroundColor: "rgba(2,6,23,0.75)",
  }}
>
  <Text style={{ color: "white", fontWeight: "900", fontSize: 12 }}>
    Move map • Confirm at center pin
  </Text>
</View>
</View>

{/* Panel */}
<View
  style={{
    position: "absolute",
    left: 12,
    right: 12,
    bottom: 88,
    backgroundColor: "rgba(2,6,23,0.92)",
    borderRadius: 20,
    padding: 14,
    borderWidth: 1,
    borderColor: "rgba(255,255,255,0.12)",
  }}
>
  <Text style={{ color: "white", fontWeight: "900", fontSize: 14, marginBottom: 10 }}>
    Report a hazard (LGU)
  </Text>

  {/* Severity */}
  <View style={{ flexDirection: "row", gap: 10, marginBottom: 10 }}>
    {SEVERITIES.map((s) => (
      <Pressable
        key={s.key}
        onPress={() => setSeverity(s.key)}
        style={{
          flex: 1,
          paddingVertical: 12,
          borderRadius: 14,
          alignItems: "center",
          backgroundColor: severity === s.key ? s.bg : "rgba(148,163,184,0.20)",
          borderWidth: 1,
          borderColor: "rgba(255,255,255,0.10)",
        }}
      >
        <Text style={{ color: "white", fontWeight: "900" }}>{s.label}</Text>
      </Pressable>
    ))}
  </View>

  {/* Type */}
  <View style={{ flexDirection: "row", flexWrap: "wrap", gap: 8, marginBottom: 12 }}>
    {HAZARD_TYPES.map((t) => {
      const active = type === t.key;
      return (
        <Pressable
          key={t.key}
          onPress={() => setType(t.key)}
          style={{
            paddingVertical: 10,

```

```

        paddingHorizontal: 12,
        borderRadius: 999,
        backgroundColor: active ? "rgba(37,99,235,0.85)" : "rgba(148,163,184,0.18)",
        borderWidth: 1,
        borderColor: "rgba(255,255,255,0.10)",
    }}
    >
    <Text style={{ color: "white", fontWeight: "800", fontSize: 12 }}>
        {t.label}
    </Text>
</Pressable>
);
    }}
</View>

<View style={{ flexDirection: "row", gap: 10 }}>
    <Pressable
        onPress={cancelPlacement}
        style={{
            flex: 1,
            backgroundColor: "rgba(148,163,184,0.22)",
            paddingVertical: 14,
            borderRadius: 14,
            alignItems: "center",
        }}
        disabled={isMutating}
    >
        <Text style={{ color: "white", fontWeight: "900" }}>CANCEL</Text>
    </Pressable>

    <Pressable
        onPress={confirmPlacement}
        style={{
            flex: 1,
            backgroundColor: "#16a34a",
            paddingVertical: 14,
            borderRadius: 14,
            alignItems: "center",
            opacity: isMutating ? 0.7 : 1,
        }}
        disabled={isMutating}
    >
        <Text style={{ color: "white", fontWeight: "900" }}>
            {isMutating ? "SAVING..." : "CONFIRM"}
        </Text>
    </Pressable>
</View>
</View>
</>
) : null;

const selectedCard = selected ? (
    <View
        style={{
            position: "absolute",
            left: 12,
            right: 12,
            bottom: 88,
            backgroundColor: "rgba(2,6,23,0.92)",
            borderRadius: 20,
            padding: 14,
            borderWidth: 1,
            borderColor: "rgba(255,255,255,0.12)",
        }}
    >
        <View style={{ flexDirection: "row", justifyContent: "space-between", alignItems: "center" }}>
            <Text style={{ color: "white", fontWeight: "900", fontSize: 14 }}>
                {shortTypeLabel((selected as any).type)} • {safeStatus(selected) === "resolved" ?
"RESOLVED" : "ACTIVE"}

```

```

</Text>

<View
  style={{
    paddingVertical: 6,
    paddingHorizontal: 10,
    borderRadius: 999,
    backgroundColor: safeStatus(selected) === "resolved" ? "rgba(148,163,184,0.18)" :
"rgba(16,185,129,0.14)",
    borderWidth: 1,
    borderColor: "rgba(255,255,255,0.12)",
  }}
  >
  <Text style={{ color: "white", fontWeight: "900", fontSize: 12 }}>
    {safeStatus(selected) === "resolved" ? "HISTORICAL" : "ACTION"}
  </Text>
</View>
</View>

<Text style={{ color: "rgba(226,232,240,0.85)", marginTop: 8, fontStyle: "italic" }}>
  {safeStatus(selected) === "resolved"
    ? "Resolved hazards are kept for historical accountability."
    : "Resolve after validation; delete only if duplicate/invalid."}
</Text>

<View style={{ flexDirection: "row", gap: 10, marginTop: 12 }}>
  <Pressable
    onPress={() => setSelectedId(null)}
    style={{
      flex: 1,
      backgroundColor: "rgba(148,163,184,0.20)",
      paddingVertical: 12,
      borderRadius: 14,
      alignItems: "center",
    }}
    disabled={isMutating}
  >
    <Text style={{ color: "white", fontWeight: "900" }}>CLOSE</Text>
  </Pressable>

  {safeStatus(selected) !== "resolved" ? (
    <Pressable
      onPress={resolveSelected}
      style={{
        flex: 1,
        backgroundColor: "#16a34a",
        paddingVertical: 12,
        borderRadius: 14,
        alignItems: "center",
        opacity: isMutating ? 0.7 : 1,
      }}
      disabled={isMutating}
    >
      <Text style={{ color: "white", fontWeight: "900" }}>
        {isMutating ? "..." : "RESOLVE"}
      </Text>
    </Pressable>
  ) : (
    <View style={{ flex: 1 }} />
  )}

  <Pressable
    onPress={deleteSelected}
    style={{
      flex: 1,
      backgroundColor: "#991b1b",
      paddingVertical: 12,
      borderRadius: 14,
      alignItems: "center",
    }}
  >

```

```

        opacity: isMutating ? 0.7 : 1,
    }}
    disabled={isMutating}
  >
    <Text style={{ color: "white", fontWeight: "900" }}>
      {isMutating ? "..." : "DELETE"}
    </Text>
  </Pressable>
</View>
</View>
) : null;

return (
  <View style={{ flex: 1 }}>
    <MapView
      ref={(r) => (mapRef.current = r)}
      style={{ flex: 1 }}
      initialRegion={region}
      onLongPress={(e: MapPressEvent) => {
        // LOCKED: long-press primary trigger
        if (!isMutating) enterPlacingMode();
      }}
      showsUserLocation={true}
      showsMyLocationButton={false}
    >
      {visibleHazards.map((h: any) => {
        const st = safeStatus(h);
        const pinColor = st === "resolved" ? "#64748b" : severityColor(h.severity);
        const opacity = st === "resolved" ? 0.25 : 1.0;

        // Remount marker when status changes to avoid reuse bleed
        const markerKey = `${h.id}-${st}`;

        return (
          <Marker
            key={markerKey}
            coordinate={{ latitude: h.latitude, longitude: h.longitude }}
            tracksViewChanges={false}
            onPress={() => onMarkerPress(String(h.id))}
          >
            {(() => {
              const m = getMarkerStyle(h.severity, st);
              return (
                <View
                  collapsable={false}
                  renderToHardwareTextureAndroid={true}
                  style={{
                    width: m.size,
                    height: m.size,
                    borderRadius: m.radius,
                    backgroundColor: st === "resolved" ? "#64748b" : severityColor(h.severity),
                    borderWidth: m.border,
                    borderColor: "white",
                    alignItems: "center",
                    justifyContent: "center",
                    opacity: m.opacity,
                    elevation: m.elevation,
                  }}
                >
                  <Text style={{ color: "white", fontWeight: "900", fontSize: 11 }}>
                    {typeGlyph(h.type)}
                  </Text>
                </View>
              );
            })()}
          </Marker>
        );
      })}
    </MapView>
  );
}

```

```

    {topBar}
    {recenterBtn}
    {placementMode ? placingOverlay : selected ? selectedCard : bottomReport}
  </View>
);
}

```

.lock_map_base_tsx.txt

```

1  import React, { useEffect, useMemo, useRef, useState } from "react";
2  import { View, Text, Pressable, Dimensions, Platform } from "react-native";
3  import MapView, { Marker, Region, MapPressEvent } from "react-native-maps";
4  import * as Location from "expo-location";
5  import {
6    fetchHazards,
7    createHazard,
8    resolveHazard,
9    deleteHazard,
10   Hazard,
11 } from "@lib/hazards";
12
13 /* ----- LGU OPS CONFIG ----- */
14
15 const DEFAULT_REGION: Region = {
16   latitude: 14.5995,
17   longitude: 120.9842,
18   latitudeDelta: 0.08,
19   longitudeDelta: 0.08,
20 };
21
22 const SEVERITIES = [
23   { key: 3, label: "HIGH", bg: "#ef4444" },
24   { key: 2, label: "MED", bg: "#f59e0b" },
25   { key: 1, label: "LOW", bg: "#10b981" },
26 ] as const;
27
28 const HAZARD_TYPES = [
29   { key: "pothole", label: "Pothole" },
30   { key: "flood", label: "Flood" },
31   { key: "crack", label: "Crack" },
32   { key: "debris", label: "Debris" },
33   { key: "construction", label: "Construction" },
34 ] as const;
35
36 type HazardType = (typeof HAZARD_TYPES)[number]["key"];
37
38 function severityColor(sev: number) {
39   if (sev === 3) return "#ef4444";
40   if (sev === 2) return "#f59e0b";
41   return "#10b981";
42 }
43
44 function safeStatus(h: any): "reported" | "resolved" {
45   return h?.status === "resolved" ? "resolved" : "reported";
46 }
47
48 function shortTypeLabel(t: string) {
49   const found = HAZARD_TYPES.find((x) => x.key === t);
50   return found ? found.label : String(t || "Hazard");
51 }
52
53 /* ----- COMPONENT ----- */
54
55 export default function LGUMap() {
56   const mapRef = useRef<MapView | null>(null);
57
58   const [region, setRegion] = useState<Region>(DEFAULT_REGION);

```

```

59   const [userCoord, setUserCoord] =
60     useState<{ latitude: number; longitude: number } | null>(null);
61
62   const [hazards, setHazards] = useState<Hazard[]>([]);
63   const [selectedId, setSelectedId] = useState<string | null>(null);
64   const [isMutating, setIsMutating] = useState(false);
65
66   // Center-pin placement mode (LOCKED)
67   const [placementMode, setPlacementMode] = useState(false);
68   const [severity, setSeverity] = useState<1 | 2 | 3>(2);
69   const [type, setType] = useState<HazardType>("pothole");
70
71   // LGU filters
72   const [severityFilter, setSeverityFilter] = useState<Record<number, boolean>>({
73     3: true,
74     2: true,
75     1: true,
76   });
77   const [typeFilter, setTypeFilter] = useState<Record<HazardType, boolean>>({
78     pothole: true,
79     flood: true,
80     crack: true,
81     debris: true,
82     construction: true,
83   });
84   const [showResolved, setShowResolved] = useState(true);
85
86   async function safeReloadHazards() {
87     try {
88       const next = await fetchHazards();
89       {
90         const arr = Array.isArray(next) ? next : [];
91         const normalized = arr.map((h:any) => ({
92           ...h,
93           severity: (h?.severity === "HIGH" ? 3 : h?.severity === "MEDIUM" ? 2 : h?.severity ===
"LOW" ? 1 : h?.severity),
94         }));
95         setHazards(normalized as any);
96       }
97     } catch (e) {
98       console.warn("[LGUMap] fetchHazards failed", e);
99       setHazards([]);
100     }
101   }
102
103   // Location bootstrap + initial load
104   useEffect(() => {
105     (async () => {
106       try {
107         const { status } = await Location.requestForegroundPermissionsAsync();
108         if (status === "granted") {
109           const loc = await Location.getLastKnownPositionAsync({});
110           if (loc?.coords) {
111             const r: Region = {
112               latitude: loc.coords.latitude,
113               longitude: loc.coords.longitude,
114               latitudeDelta: 0.06,
115               longitudeDelta: 0.06,
116             };
117             setUserCoord({ latitude: r.latitude, longitude: r.longitude });
118             setRegion(r);
119           }
120         }
121       } catch {
122         // keep default region
123       }
124       await safeReloadHazards();
125     })();
126   }, []);

```

```

127
128 const selected = useMemo(() => {
129     if (!selectedId) return null;
130     return hazards.find((h: any) => String(h.id) === String(selectedId)) ?? null;
131 }, [selectedId, hazards]);
132
133 // Clear selection if it disappears
134 useEffect(() => {
135     if (selectedId && !hazards.some((h: any) => String(h.id) === String(selectedId))) {
136         setSelectedId(null);
137     }
138 }, [hazards, selectedId]);
139
140 // LOCKED: single source of truth
141 function enterPlacingMode() {
142     setSelectedId(null);
143     setPlacementMode(true);
144 }
145
146 function cancelPlacement() {
147     setPlacementMode(false);
148 }
149
150 async function confirmPlacement() {
151     if (!mapRef.current || isMutating) return;
152
153     try {
154         setIsMutating(true);
155
156         const { width, height } = Dimensions.get("window");
157         // Best-effort: center of the screen. Good enough for ops.
158         const coord = region;
159
160         await createHazard({
161             latitude: coord.latitude,
162             longitude: coord.longitude,
163             severity,
164             type,
165         });
166
167         await safeReloadHazards();
168     } catch (e) {
169         console.warn("[LGUMap] confirmPlacement failed", e);
170     } finally {
171         setIsMutating(false);
172         setPlacementMode(false);
173     }
174 }
175
176 function onMarkerPress(id: string) {
177     if (isMutating || placementMode) return;
178     setSelectedId(id);
179 }
180
181 async function resolveSelected() {
182     if (!selected || isMutating) return;
183     try {
184         setIsMutating(true);
185         await resolveHazard(String((selected as any).id));
186         await safeReloadHazards();
187     } catch (e) {
188         console.warn("[LGUMap] resolveSelected failed", e);
189     } finally {
190         setIsMutating(false);
191     }
192 }
193
194 async function deleteSelected() {
195     if (!selected || isMutating) return;

```

```

196
197     const id = String((selected as any).id);
198     setSelectedId(null);
199
200     try {
201         setIsMutating(true);
202         await deleteHazard(id);
203         await safeReloadHazards();
204     } catch (e) {
205         console.warn("[LGUMap] deleteSelected failed", e);
206         await safeReloadHazards();
207     } finally {
208         setIsMutating(false);
209     }
210 }
211
212 function recenter() {
213     if (!mapRef.current || !userCoord) return;
214
215     const r: Region = {
216         latitude: userCoord.latitude,
217         longitude: userCoord.longitude,
218         latitudeDelta: 0.06,
219         longitudeDelta: 0.06,
220     };
221
222     setRegion(r);
223     mapRef.current.animateToRegion(r, 350);
224 }
225
226 const hazardsStable = useMemo(() => {
227     const arr = [...hazards] as any[];
228     arr.sort((a, b) => String(a.id).localeCompare(String(b.id)));
229     return arr;
230 }, [hazards]);
231
232 const visibleHazards = useMemo(() => {
233     return hazardsStable.filter((h: any) => {
234         const st = safeStatus(h);
235         if (!showResolved && st === "resolved") return false;
236         if (!severityFilter[h.severity]) return false;
237         if (!typeFilter[h.type as HazardType]) return false;
238         return true;
239     });
240 }, [hazardsStable, severityFilter, typeFilter, showResolved]);
241
242 const counts = useMemo(() => {
243     let high = 0, med = 0, low = 0;
244     let active = 0, resolved = 0;
245
246     for (const h of hazardsStable as any[]) {
247         const st = safeStatus(h);
248         if (st === "resolved") resolved++;
249         else active++;
250
251         if (h.severity === 3) high++;
252         else if (h.severity === 2) med++;
253         else low++;
254     }
255     return { high, med, low, active, resolved };
256 }, [hazardsStable]);
257
258 const topBar = (
259     <View
260         style={{
261             position: "absolute",
262             top: 12,
263             left: 12,
264             right: 12,

```

```

265         padding: 10,
266         borderRadius: 14,
267         backgroundColor: "rgba(2,6,23,0.74)",
268         borderWidth: 1,
269         borderColor: "rgba(255,255,255,0.12)",
270     }}
271     >
272     <View style={{ flexDirection: "row", alignItems: "center", justifyContent: "space-
between" }}>
273         <Text style={{ color: "white", fontWeight: "900", letterSpacing: 0.4 }}>
274             LGU HAZARD OPS
275         </Text>
276
277         <Pressable
278             onPress={() => setShowResolved((v) => !v)}
279             style={{
280                 paddingVertical: 8,
281                 paddingHorizontal: 12,
282                 borderRadius: 999,
283                 backgroundColor: showResolved ? "rgba(148,163,184,0.22)" :
"rgba(239,68,68,0.18)",
284                 borderWidth: 1,
285                 borderColor: "rgba(255,255,255,0.15)",
286             }}
287         >
288             <Text style={{ color: "white", fontWeight: "900", fontSize: 12 }}>
289                 {showResolved ? "Resolved: ON" : "Resolved: OFF"}
290             </Text>
291         </Pressable>
292     </View>
293
294     <View style={{ flexDirection: "row", gap: 8, marginTop: 10 }}>
295         <View style={{ flex: 1, borderRadius: 12, padding: 10, backgroundColor:
"rgba(239,68,68,0.16)", borderWidth: 1, borderColor: "rgba(239,68,68,0.22)" }}>
296             <Text style={{ color: "white", fontWeight: "900", fontSize: 12 }}>HIGH</Text>
297             <Text style={{ color: "white", fontWeight: "900", fontSize: 18
}}>{counts.high}</Text>
298         </View>
299         <View style={{ flex: 1, borderRadius: 12, padding: 10, backgroundColor:
"rgba(245,158,11,0.14)", borderWidth: 1, borderColor: "rgba(245,158,11,0.20)" }}>
300             <Text style={{ color: "white", fontWeight: "900", fontSize: 12 }}>MED</Text>
301             <Text style={{ color: "white", fontWeight: "900", fontSize: 18
}}>{counts.med}</Text>
302         </View>
303         <View style={{ flex: 1, borderRadius: 12, padding: 10, backgroundColor:
"rgba(16,185,129,0.14)", borderWidth: 1, borderColor: "rgba(16,185,129,0.20)" }}>
304             <Text style={{ color: "white", fontWeight: "900", fontSize: 12 }}>LOW</Text>
305             <Text style={{ color: "white", fontWeight: "900", fontSize: 18
}}>{counts.low}</Text>
306         </View>
307     </View>
308
309     <View style={{ flexDirection: "row", gap: 8, marginTop: 10 }}>
310         <Text style={{ color: "rgba(226,232,240,0.85)", fontWeight: "800", fontSize: 12 }}>
311             Active: {counts.active} • Resolved: {counts.resolved}
312         </Text>
313
314         <Pressable
315             onPress={safeReloadHazards}
316             style={{
317                 marginLeft: "auto",
318                 paddingVertical: 6,
319                 paddingHorizontal: 10,
320                 borderRadius: 999,
321                 backgroundColor: "rgba(37,99,235,0.18)",
322                 borderWidth: 1,
323                 borderColor: "rgba(37,99,235,0.22)",
324             }}
325             disabled={isMutating}

```

```

326     >
327     <Text style={{ color: "white", fontWeight: "900", fontSize: 12 }}>
328         REFRESH
329     </Text>
330 </Pressable>
331 </View>
332 </View>
333 );
334
335 const bottomReport = (
336     <View style={{ position: "absolute", left: 14, right: 14, bottom: 88 }}>
337         <Pressable
338             onPress={enterPlacingMode}
339             style={{
340                 backgroundColor: "#2563eb",
341                 paddingVertical: 16,
342                 borderRadius: 18,
343                 alignItems: "center",
344                 shadowColor: "#000",
345                 shadowOpacity: 0.25,
346                 shadowRadius: 10,
347                 shadowOffset: { width: 0, height: 6 },
348                 elevation: 6,
349                 opacity: isMutating ? 0.7 : 1,
350             }}
351             disabled={isMutating}
352         >
353             <Text style={{ color: "white", fontWeight: "900", fontSize: 16, letterSpacing: 0.3
354                 + REPORT HAZARD
355             </Text>
356             <Text style={{ color: "rgba(226,232,240,0.85)", fontWeight: "800", fontSize: 12,
marginTop: 6 }}>
357                 Long-press also works (primary)
358             </Text>
359         </Pressable>
360     </View>
361 );
362
363 const recenterBtn = (
364     <Pressable
365         onPress={recenter}
366         style={{
367             position: "absolute",
368             right: 14,
369             bottom: 166,
370             paddingVertical: 10,
371             paddingHorizontal: 12,
372             borderRadius: 18,
373             backgroundColor: "rgba(0,0,0,0.85)",
374             borderWidth: 1,
375             borderColor: "rgba(255,255,255,0.22)",
376             flexDirection: "row",
377             gap: 10,
378             alignItems: "center",
379             justifyContent: "center",
380             shadowColor: "#000",
381             shadowOpacity: 0.30,
382             shadowRadius: 12,
383             shadowOffset: { width: 0, height: 8 },
384             elevation: 9,
385             opacity: userCoord ? 1 : 0.45,
386         }}
387         disabled={!userCoord}
388     >
389     <View
390         style={{
391             width: 42,
392             height: 42,

```

```

393         borderRadius: 21,
394         backgroundColor: "rgba(37,99,235,0.95)",
395         alignItems: "center",
396         justifyContent: "center",
397     }}
398 >
399     <Text style={{ color: "white", fontSize: 22, fontWeight: "900" }}>■</Text>
400 </View>
401
402 <View style={{ alignItems: "flex-start" }}>
403     <Text style={{ color: "white", fontWeight: "900", fontSize: 12, letterSpacing: 0.2
}}>
404         RECENTER
405     </Text>
406     <Text style={{ color: "rgba(226,232,240,0.85)", fontWeight: "800", fontSize: 11 }}>
407         My location
408     </Text>
409 </View>
410 </Pressable>
411 );
412
413 const placingOverlay = placementMode ? (
414     <>
415         { /* Center pin */ }
416         <View
417             pointerEvents="none"
418             style={{
419                 position: "absolute",
420                 left: 0,
421                 right: 0,
422                 top: 0,
423                 bottom: 0,
424                 alignItems: "center",
425                 justifyContent: "center",
426             }}
427         >
428             <View
429                 style={{
430                     width: 18,
431                     height: 18,
432                     borderRadius: 9,
433                     backgroundColor: "#dc2626",
434                     borderWidth: 3,
435                     borderColor: "white",
436                     shadowColor: "#000",
437                     shadowOpacity: 0.25,
438                     shadowRadius: 8,
439                     shadowOffset: { width: 0, height: 4 },
440                     elevation: 6,
441                 }}
442             />
443             <View
444                 style={{
445                     marginTop: 10,
446                     paddingHorizontal: 12,
447                     paddingVertical: 8,
448                     borderRadius: 999,
449                     backgroundColor: "rgba(2,6,23,0.75)",
450                 }}
451             >
452                 <Text style={{ color: "white", fontWeight: "900", fontSize: 12 }}>
453                     Move map • Confirm at center pin
454                 </Text>
455             </View>
456         </View>
457
458         { /* Panel */ }
459         <View
460             style={{

```

```

461         position: "absolute",
462         left: 12,
463         right: 12,
464         bottom: 88,
465         backgroundColor: "rgba(2,6,23,0.92)",
466         borderRadius: 20,
467         padding: 14,
468         borderWidth: 1,
469         borderColor: "rgba(255,255,255,0.12)",
470     }}
471 >
472     <Text style={{ color: "white", fontWeight: "900", fontSize: 14, marginBottom: 10 }}>
473         Report a hazard (LGU)
474     </Text>
475
476     {/* Severity */}
477     <View style={{ flexDirection: "row", gap: 10, marginBottom: 10 }}>
478         {SEVERITIES.map((s) => (
479             <Pressable
480                 key={s.key}
481                 onPress={() => setSeverity(s.key)}
482                 style={{
483                     flex: 1,
484                     paddingVertical: 12,
485                     borderRadius: 14,
486                     alignItems: "center",
487                     backgroundColor: severity === s.key ? s.bg : "rgba(148,163,184,0.20)",
488                     borderWidth: 1,
489                     borderColor: "rgba(255,255,255,0.10)",
490                 }}
491             >
492                 <Text style={{ color: "white", fontWeight: "900" }}>{s.label}</Text>
493             </Pressable>
494         ))}
495     </View>
496
497     {/* Type */}
498     <View style={{ flexDirection: "row", flexWrap: "wrap", gap: 8, marginBottom: 12 }}>
499         {HAZARD_TYPES.map((t) => {
500             const active = type === t.key;
501             return (
502                 <Pressable
503                     key={t.key}
504                     onPress={() => setType(t.key)}
505                     style={{
506                         paddingVertical: 10,
507                         paddingHorizontal: 12,
508                         borderRadius: 99,
509                         backgroundColor: active ? "rgba(37,99,235,0.85)" :
"rgba(148,163,184,0.18)",
510                         borderWidth: 1,
511                         borderColor: "rgba(255,255,255,0.10)",
512                     }}
513                 >
514                     <Text style={{ color: "white", fontWeight: "800", fontSize: 12 }}>
515                         {t.label}
516                     </Text>
517                 </Pressable>
518             );
519         })}
520     </View>
521
522     <View style={{ flexDirection: "row", gap: 10 }}>
523         <Pressable
524             onPress={cancelPlacement}
525             style={{
526                 flex: 1,
527                 backgroundColor: "rgba(148,163,184,0.22)",
528                 paddingVertical: 14,

```

```

529         borderRadius: 14,
530         alignItems: "center",
531     }}
532     disabled={isMutating}
533 >
534     <Text style={{ color: "white", fontWeight: "900" }}>CANCEL</Text>
535 </Pressable>
536
537     <Pressable
538         onPress={confirmPlacement}
539         style={{
540             flex: 1,
541             backgroundColor: "#16a34a",
542             paddingVertical: 14,
543             borderRadius: 14,
544             alignItems: "center",
545             opacity: isMutating ? 0.7 : 1,
546         }}
547         disabled={isMutating}
548     >
549         <Text style={{ color: "white", fontWeight: "900" }}>
550             {isMutating ? "SAVING..." : "CONFIRM"}
551         </Text>
552     </Pressable>
553 </View>
554 </View>
555 </>
556 ) : null;
557
558 const selectedCard = selected ? (
559     <View
560         style={{
561             position: "absolute",
562             left: 12,
563             right: 12,
564             bottom: 88,
565             backgroundColor: "rgba(2,6,23,0.92)",
566             borderRadius: 20,
567             padding: 14,
568             borderWidth: 1,
569             borderColor: "rgba(255,255,255,0.12)",
570         }}
571     >
572         <View style={{ flexDirection: "row", justifyContent: "space-between", alignItems:
"center" }}>
573             <Text style={{ color: "white", fontWeight: "900", fontSize: 14 }}>
574                 {shortTypeLabel((selected as any).type)} • {safeStatus(selected) === "resolved" ?
"RESOLVED" : "ACTIVE"}
575             </Text>
576
577             <View
578                 style={{
579                     paddingVertical: 6,
580                     paddingHorizontal: 10,
581                     borderRadius: 999,
582                     backgroundColor: safeStatus(selected) === "resolved" ? "rgba(148,163,184,0.18)"
: "rgba(16,185,129,0.14)",
583                     borderWidth: 1,
584                     borderColor: "rgba(255,255,255,0.12)",
585                 }}
586             >
587                 <Text style={{ color: "white", fontWeight: "900", fontSize: 12 }}>
588                     {safeStatus(selected) === "resolved" ? "HISTORICAL" : "ACTION"}
589                 </Text>
590             </View>
591         </View>
592
593         <Text style={{ color: "rgba(226,232,240,0.85)", marginTop: 8, fontStyle: "italic" }}>
594             {safeStatus(selected) === "resolved"

```

```

595     ? "Resolved hazards are kept for historical accountability."
596     : "Resolve after validation; delete only if duplicate/invalid."
597   </Text>
598
599   <View style={{ flexDirection: "row", gap: 10, marginTop: 12 }}>
600     <Pressable
601       onPress={() => setSelectedId(null)}
602       style={{
603         flex: 1,
604         backgroundColor: "rgba(148,163,184,0.20)",
605         paddingVertical: 12,
606         borderRadius: 14,
607         alignItems: "center",
608       }}
609       disabled={isMutating}
610     >
611       <Text style={{ color: "white", fontWeight: "900" }}>CLOSE</Text>
612     </Pressable>
613
614     {safeStatus(selected) !== "resolved" ? (
615       <Pressable
616         onPress={resolveSelected}
617         style={{
618           flex: 1,
619           backgroundColor: "#16a34a",
620           paddingVertical: 12,
621           borderRadius: 14,
622           alignItems: "center",
623           opacity: isMutating ? 0.7 : 1,
624         }}
625         disabled={isMutating}
626       >
627         <Text style={{ color: "white", fontWeight: "900" }}>
628           {isMutating ? "..." : "RESOLVE"}
629         </Text>
630       </Pressable>
631     ) : (
632       <View style={{ flex: 1 }} />
633     )
634
635     <Pressable
636       onPress={deleteSelected}
637       style={{
638         flex: 1,
639         backgroundColor: "#991b1b",
640         paddingVertical: 12,
641         borderRadius: 14,
642         alignItems: "center",
643         opacity: isMutating ? 0.7 : 1,
644       }}
645       disabled={isMutating}
646     >
647       <Text style={{ color: "white", fontWeight: "900" }}>
648         {isMutating ? "..." : "DELETE"}
649       </Text>
650     </Pressable>
651   </View>
652 </View>
653 ) : null;
654
655 return (
656   <View style={{ flex: 1 }}>
657     <MapView onRegionChangeComplete={setRegion}
658       ref={(r) => (mapRef.current = r)}
659       style={{ flex: 1 }}
660       initialRegion={region}
661       onLongPress={(e: MapPressEvent) => {
662         // LOCKED: long-press primary trigger
663         if (!isMutating) enterPlacingMode();

```

```

664     }}
665     showsUserLocation={true}
666     showsMyLocationButton={false}
667   >
668     {visibleHazards.map((h: any) => {
669       const st = safeStatus(h);
670       const pinColor = st === "resolved" ? "#64748b" : severityColor(h.severity);
671       const opacity = st === "resolved" ? 0.25 : 1.0;
672
673       // Remount marker when status changes to avoid reuse bleed
674       const markerKey = `${h.id}-${st}`;
675
676       return (
677         <Marker
678           key={markerKey}
679           coordinate={{ latitude: h.latitude, longitude: h.longitude }}
680           pinColor={pinColor}
681           opacity={opacity}
682           tracksViewChanges={false}
683           onPress={() => onMarkerPress(String(h.id))}
684         />
685       );
686     })}
687   </MapView>
688
689   {topBar}
690   {recenterBtn}
691   {placementMode ? placingOverlay : selected ? selectedCard : bottomReport}
692 </View>
693 );
694 }

```

.lock_placement_flow.txt

```

7: createHazard,
141: function enterPlacingMode() {
150: async function confirmPlacement() {
160:   await createHazard({
169:     console.warn("[LGUMap] confirmPlacement failed", e);
338:     onPress={enterPlacingMode}
538:     onPress={confirmPlacement}
663:     if (!isMutating) enterPlacingMode();

```

.lock_severity_flow.txt

```

25: { key: 1, label: "LOW", bg: "#10b981" },
86: async function safeReloadHazards() {
91: const normalized = arr.map((h:any) => ({
93:   severity: (h?.severity === "HIGH" ? 3 : h?.severity === "MEDIUM" ? 2 : h?.severity === "LOW"
? 1 : h?.severity),
95: setHazards(normalized as any);
124:   await safeReloadHazards();
167:   await safeReloadHazards();
186:   await safeReloadHazards();
203:   await safeReloadHazards();
206:   await safeReloadHazards();
304:   <Text style={{ color: "white", fontWeight: "900", fontSize: 12 }}>LOW</Text>
315:   onPress={safeReloadHazards}

```

.lock_supabase_constraints.txt

```
CHECK (severity IN ('LOW', 'MEDIUM', 'HIGH'))
```

.lock_supabase_severity.txt

Distinct severity values (sampled up to 2000):
{ MEDIUM: 48, LOW: 2, HIGH: 7 }